

Automatas, Teoria de Lenguajes y Compiladores

Trabajo Practico

Primera Entrega – Diseño del Lenguaje

02/09/2026

1 Equipo

Nombre	Apellido	Legajo	E-mail
Geronimo	Naso Rodriguez	64177	gnasorodriguez@itba.edu.ar
Juan Ignacio	Garcia Vautrin Raggio	63319	jgarciaautrinraggi@itba.edu.ar
Santiago	Fernandez Pacheco	65868	sfernandezpacheco@itba.edu.ar
Manuel	Blacker	65184	mblacker@itba.edu.ar

Link al repositorio: <https://github.com/JuaniRaggio/ccc>

2 Dominio

El proyecto consiste en un compilador que traduce un subconjunto de C a SystemVerilog sintetizable, con énfasis en la detección y explotación de paralelismo a nivel de hardware. El objetivo no es solo traducir sintaxis, sino analizar dependencias de datos entre operaciones y generar hardware paralelo cuando las operaciones son independientes entre sí.

El dominio de aplicación es la síntesis de alto nivel (High-Level Synthesis): el programador describe su algoritmo en C, un lenguaje secuencial que ya conoce, y el compilador se encarga de determinar que partes pueden ejecutarse en paralelo en hardware real (FPGAs o ASICs). Esto elimina la necesidad de escribir Verilog a mano, que es considerablemente más verboso y requiere pensar directamente en términos de ciclos de clock, registros y señales.

El compilador acepta como entrada un archivo `.c` con anotaciones opcionales y produce como salida un módulo SystemVerilog sintetizable con su correspondiente testbench.

2.1 Alcance del subconjunto de C soportado

El compilador soporta las siguientes construcciones del lenguaje C:

- Tipos enteros con ancho de bits explícito: `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`
- Tipo booleano: `bool`
- Arreglos unidimensionales de tipos enteros

- Expresiones aritmeticas: `+`, `-`, `*`
- Expresiones logicas y de comparacion: `&&`, `||`, `!`, `==`, `!=`, `<`, `>`, `<=`, `>=`
- Estructuras de control: `if / else`, `for`, `while`
- Funciones con parametros y valor de retorno (cada funcion se traduce a un modulo Verilog independiente)
- Referencias como parametros de funcion con qualifiers de aliasing: `unique` (default, garantia de no-aliasing) y `aliased` (puede referirse al mismo objeto que otra referencia)
- Anotacion `__parallel` para marcar bloques que el programador garantiza como independientes
- Builtins del lenguaje para operaciones matematicas comunes: `__abs`, `__min`, `__max`
- Variables `constexpr`: constantes evaluadas en tiempo de compilacion, se traducen a `localparam` en SystemVerilog
- Funciones `constexpr`: funciones evaluadas completamente en tiempo de compilacion, no generan ningun modulo de hardware

No se soportan: memoria dinamica, recursion, tipos de punto flotante, ni llamadas a funciones de biblioteca estandar. Las funciones de `stdlib` no tienen analogo en hardware sintetizable ya que asumen la existencia de sistema operativo, heap y file descriptors. En su lugar el lenguaje provee builtins con semantica de hardware definida.

Las referencias son validas unicamente como parametros de funcion. A diferencia de los punteros no pueden ser nulas ni reasignadas, lo que elimina toda una clase de errores que no tienen sentido en hardware sintetizable: en hardware una referencia modela una conexion de cable que siempre existe. Por defecto todas las referencias son `unique`: el compilador asume no-aliasing y puede paralelizar operaciones sobre ellas. Si el programador necesita expresar que dos referencias pueden referirse al mismo objeto, debe anotarlas con `aliased`, lo que deshabilita la paralelizacion automatica entre esas variables.

3 Construcciones

3.1 Construcciones de entrada (C)

3.1.1 Declaracion de funcion

Cada funcion de nivel superior se traduce a un modulo SystemVerilog independiente. Los parametros se convierten en puertos de entrada y el valor de retorno en un puerto de salida.

```
int32_t suma(int32_t a, int32_t b) {
    return a + b;
}
```

3.1.2 Referencias con qualifier de aliasing

Las referencias son validas unicamente como parametros de funcion. El qualifier va entre el `&` y el nombre del parametro, consistente con como C trata `const`.

`unique` es el default e indica que el compilador puede asumir que esa referencia no se solapa con ninguna otra. `aliased` desactiva esa garantia. Al no poder ser nulas, el compilador no necesita emitir chequeos de null, lo que simplifica el hardware generado.

```
// unique es el default, estas dos firmas son equivalentes
void escalar(int32_t &unique salida, int32_t &unique entrada, int32_t factor);
void escalar(int32_t &salida, int32_t &entrada, int32_t factor);

// aliased: el compilador no puede asumir que salida != entrada
void in_place(int32_t &aliased salida, int32_t &entrada, int32_t factor);
```

El compilador detecta en el call site los casos obvios de aliasing (pasar el mismo simbolo dos veces a parametros `unique`) y emite un error.

Esta decision representa una mejora significativa respecto a C estandar. En C, cualquier puntero puede ser `NULL` y el programador es responsable de verificarlo antes de cada uso. Esos chequeos generan branches adicionales que en hardware se traducen en logica extra y potenciales ciclos de clock desperdiciados. Al usar referencias, la garantia de no-nulidad es estatica y verificada en tiempo de compilacion, por lo que el hardware generado no contiene ninguna logica de validacion de direcciones.

3.1.3 Builtins del lenguaje

El lenguaje no soporta llamadas a funciones de biblioteca estandar porque estas asumen la existencia de sistema operativo, heap y file descriptors, ninguno de los cuales existe en hardware sintetizable. En su lugar se proveen builtins con semantica de hardware definida por el compilador:

Builtin	Descripcion	Hardware generado
<code>__abs(x)</code>	Valor absoluto	Logica combinacional
<code>__min(x, y)</code>	Minimo entre dos valores	Comparador + multiplexor
<code>__max(x, y)</code>	Maximo entre dos valores	Comparador + multiplexor

```
int32_t normalizar(int32_t x, int32_t tope) {
    return __min(__abs(x), tope);
}
```

3.1.4 Bloque paralelo

La anotacion `__parallel` indica al compilador que las sentencias dentro del bloque no tienen dependencias entre si y pueden sintetizarse como logica combinacional concurrente.

```
__parallel {
    resultado_a = f(x);
    resultado_b = g(y);
}
```

3.1.5 Bucle `for` con rango estatico

Los bucles `for` con limites conocidos en tiempo de compilacion se pueden desenrollar o sintetizar como maquinas de estados finitos (FSM) con pipeline.

```
for (int32_t i = 0; i < 8; i++) {  
    acum += datos[i];  
}
```

3.1.6 Condicional `if` / `else`

Se traduce a un multiplexor en logica combinacional o a una transicion de estado en una FSM.

```
if (x > umbral) {  
    salida = x - umbral;  
} else {  
    salida = 0;  
}
```

3.1.7 Variables y funciones `constexpr`

Las variables `constexpr` son constantes evaluadas en tiempo de compilacion. Se traducen a `localparam` en SystemVerilog y pueden usarse como limites de bucles, tamaños de arreglos o cualquier lugar donde el hardware requiere un valor estatico.

```
constexpr int32_t N = 8;  
constexpr int32_t MAX_VAL = (1 << N) - 1;  
constexpr int32_t TABLA[4] = {1, 2, 4, 8};
```

Las funciones `constexpr` son funciones cuya ejecucion ocurre enteramente en tiempo de compilacion. No generan ningun modulo de hardware: el compilador las evalua y sustituye el resultado como si fuera un literal. Solo pueden recibir y retornar valores `constexpr`, y su cuerpo esta restringido a las mismas construcciones del lenguaje exceptuando referencias.

```
constexpr int32_t potencia(int32_t base, int32_t exp) {  
    int32_t r = 1;  
    for (int32_t i = 0; i < exp; i++)  
        r *= base;  
    return r;  
}  
  
constexpr int32_t LUT[4] = {  
    potencia(2, 0),  
    potencia(2, 1),  
    potencia(2, 2),  
    potencia(2, 3),  
};
```

Esto resuelve el caso de bucles con limite aparentemente dinamico: si el limite es una variable `constexpr`, el compilador lo conoce en tiempo de compilacion y puede sintetizar la FSM correctamente.

```
constexpr int32_t N = 16;
```

```
int32_t sumar(int32_t &datos) {
    int32_t acum = 0;
    for (int32_t i = 0; i < N; i++) // N es constexpr: aceptado
        acum += datos[i];
    return acum;
}
```

3.2 Construcciones de salida (SystemVerilog)

3.2.1 Modulo

Cada funcion C se convierte en un modulo con puertos `clk`, `rst`, `start`, `done`, los parametros como entradas y el retorno como salida.

3.2.2 `always_comb`

Para expresiones puramente combinacionales sin estado: asignaciones directas, operaciones aritmeticas simples, multiplexores.

3.2.3 `always_ff`

Para logica secuencial: bucles que requieren multiples ciclos, acumuladores, FSMs de control.

3.2.4 Bloques `always` paralelos

Cuando el compilador detecta (o el programador anota con `__parallel`) que dos computos son independientes, genera dos bloques `always_comb` separados que el sintetizador ejecuta en paralelo en hardware.

4 Casos de Prueba

4.1 Casos de aceptacion

El compilador debe aceptar y traducir correctamente los siguientes programas.

N	Descripcion	Construccion cubierta
A1	Suma de dos enteros de 32 bits	Expresion aritmetica, <code>always_comb</code>
A2	Clasificacion de un valor respecto a un umbral	<code>if / else</code> , multiplexor
A3	Suma acumulada de un arreglo de 8 elementos	Bucle <code>for</code> , FSM, <code>always_ff</code>
A4	Calculo de dos funciones independientes sobre la misma entrada	<code>__parallel</code> , bloques concurrentes
A5	Pipeline: filtrar y luego acumular un arreglo	Composicion de modulos, latencia
A6	Funcion con condicional dentro de un bucle	Combinacion de FSM y multiplexor
A7	Dos referencias <code>unique</code> a variables distintas	Paralelizacion correcta con <code>unique</code>
A8	Referencia <code>aliased</code> usada correctamente sin paralelismo	<code>aliased</code> deshabilita paralelizacion

A9	Uso de <code>__abs</code> , <code>__min</code> , <code>__max</code> en expresion	Builtins del lenguaje
A10	Bucle <code>for</code> con limite conocido en tiempo de compilacion	Rango estatico, FSM o desenrollado
A11	Variable <code>constexpr</code> usada como limite de bucle	<code>constexpr</code> , <code>localparam</code> , rango estatico
A12	Funcion <code>constexpr</code> usada para inicializar un arreglo <code>constexpr</code>	Evaluacion en tiempo de compilacion

4.2 Casos de rechazo

El compilador debe rechazar los siguientes programas con un mensaje de error descriptivo.

N	Descripcion	Razon del rechazo
R1	Funcion recursiva	No se soporta recursion
R2	Variable de tipo <code>float</code> o <code>double</code>	Tipos de punto flotante no soportados
R3	Llamada a <code>printf</code> , <code>malloc</code> u otra funcion de <code>stdlib</code>	Funciones de biblioteca estandar no soportadas
R4	Referencia declarada como variable local (no como parametro)	Referencias validas solo como parametros
R5	Pasar el mismo simbolo dos veces a parametros <code>unique</code>	Aliasing detectado en call site
R6	Bucle <code>for</code> con limite que no es literal ni <code>constexpr</code>	Rango dinamico no sintetizable como FSM estatica
R9	Funcion <code>constexpr</code> que recibe una referencia como parametro	Las funciones <code>constexpr</code> no pueden operar sobre referencias
R10	Funcion <code>constexpr</code> que llama a una funcion no <code>constexpr</code>	Solo puede llamar otras funciones <code>constexpr</code>
R7	Bloque <code>__parallel</code> con sentencias que comparten una variable escrita	Dependencia de datos dentro del bloque paralelo
R8	Funcion sin tipo de retorno explicito	Toda funcion debe declarar su tipo de retorno

5 Ejemplos

5.1 Operacion combinacional simple

Dos operaciones independientes sobre la misma entrada se sintetizan en paralelo. El compilador detecta que `cuadrado` y `valor_absoluto` no comparten escrituras y genera dos bloques `always_comb` separados.

Entrada (C)

```
__parallel {
    cuadrado = x * x;
```

Salida (SystemVerilog)

```
always_comb begin
    cuadrado = x * x;
end
```

```
absoluto = x < 0 ? -x : x;
}
```

```
always_comb begin
    if (x < 0)
        absoluto = -x;
    else
        absoluto = x;
end
```

5.2 Condicional como multiplexor

Un `if/else` simple sin estado se traduce directamente a logica combinacional con un multiplexor implicito.

Entrada (C)

```
int32_t clamp(int32_t x, int32_t tope) {
    if (x > tope)
        return tope;
    else
        return x;
}
```

Salida (SystemVerilog)

```
module clamp (
    input  logic signed [31:0] x,
    input  logic signed [31:0] tope,
    output logic signed [31:0] out
);
    always_comb begin
        if (x > tope)
            out = tope;
        else
            out = x;
    end
endmodule
```

5.3 Bucle con acumulador (FSM)

Un bucle `for` con un acumulador requiere estado entre ciclos de clock. El compilador genera una FSM con estados `IDLE`, `COMPUTE` y `DONE`.

Entrada (C)

```
int32_t sumar(int32_t datos[8]) {
    int32_t acum = 0;
    for (int32_t i = 0; i < 8; i++)
        acum += datos[i];
    return acum;
}
```

Salida (SystemVerilog)

```
// FSM con estados:
// IDLE -> COMPUTE -> DONE
always_ff @(posedge clk) begin
    case (estado)
        IDLE: begin
            acum  <= 0;
            i     <= 0;
            if (start) estado <= COMPUTE;
        end
        COMPUTE: begin
            acum  <= acum + datos[i];
            i     <= i + 1;
            if (i == 7) estado <= DONE;
        end
        DONE: begin
            done <= 1;
            out  <= acum;
        end
    endcase
end
```

```

        estado <= IDLE;
    end
endcase
end

```

5.4 Hardware parametrizable con `constexpr`

Una funcion `constexpr` calcula una lookup table de potencias de dos en tiempo de compilacion. El compilador evalua `potencia` completamente antes de generar hardware: el arreglo `LUT` se convierte en un `localparam` con valores literales y la funcion `potencia` no genera ningun modulo. El bucle usa `N` como limite, que al ser `constexpr` es aceptado y sintetizado como FSM estatica.

Entrada (C)

```

constexpr int32_t N = 4;

constexpr int32_t potencia(
    int32_t base,
    int32_t exp
) {
    int32_t r = 1;
    for (int32_t i = 0; i < exp; i++)
        r *= base;
    return r;
}

constexpr int32_t LUT[N] = {
    potencia(2, 0),
    potencia(2, 1),
    potencia(2, 2),
    potencia(2, 3),
};

int32_t buscar(int32_t idx) {
    return LUT[idx];
}

```

Salida (SystemVerilog)

```

// potencia y LUT resueltos en
// tiempo de compilacion.
// No se genera hardware para ellos.

localparam int N = 4;
localparam int LUT [0:3] = '{
    1, 2, 4, 8
};

module buscar (
    input  logic signed [31:0] idx,
    output logic signed [31:0] out
);
    always_comb begin
        out = LUT[idx];
    end
endmodule

```

5.5 Rechazo: funcion `constexpr` recursiva

Las funciones `constexpr` no pueden ser recursivas. La evaluacion en tiempo de compilacion requiere que el compilador pueda determinar un orden de computo finito y estatico. Una funcion recursiva no garantiza terminacion sin analisis adicional, lo que esta fuera del alcance del compilador.

```

// ERROR: constexpr function 'factorial' is recursive
constexpr int32_t factorial(int32_t n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

```

El compilador rechaza este programa con un error en tiempo de compilacion. La version correcta reemplaza la recursion por un bucle iterativo, que si es valido dentro de una funcion `constexpr`:


```
// Correcto: iterativo, terminacion garantizada
constexpr int32_t factorial(int32_t n) {
    int32_t r = 1;
    for (int32_t i = 2; i <= n; i++)
        r *= i;
    return r;
}
```